

ENGSE206 · การกำหนดความต้องการและการออกแบบทางซอฟต์แวร์

วิศวกรรมซอฟต์แวร์ · มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี (ดอยสะเก็ด)

Use Case Diagram

ทฤษฎี หลักการพื้นฐาน และการแปลงจากสถานการณ์จริง

เอกสารประกอบการสอนเชิงลึก — เจาะเฉพาะเรื่อง Use Case Diagram



ทีมนักศึกษาเรียงการ์ด flow เพื่อวิเคราะห์ actor และ use case

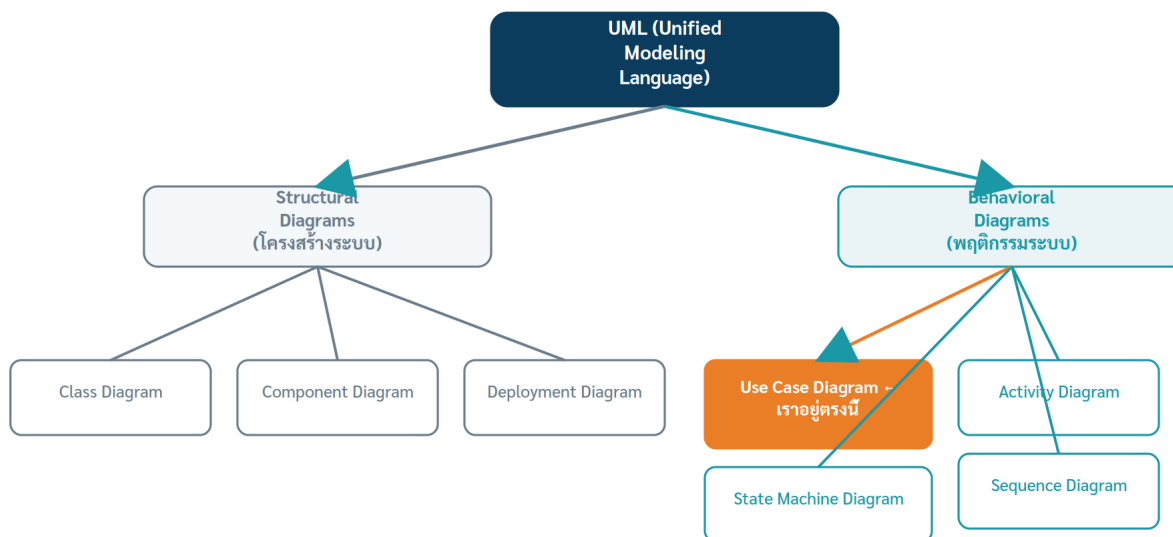
คำถามหลักของเอกสารนี้

เริ่มต้นจากศูนย์ เราจะรู้ได้อย่างไรว่าระบบหนึ่ง ๆ มี actor และ use case อะไรบ้าง แล้ววาดออกมาเป็น Use Case Diagram ที่ถูกต้องตามหลัก UML ได้อย่างไร?

1 Use Case Diagram มาจากไหน (UML)

UML (Unified Modeling Language) คือชุดสัญลักษณ์มาตรฐานสำหรับสร้างแบบจำลองระบบซอฟต์แวร์

พัฒนาขึ้นในช่วงกลางทศวรรษ 1990 โดย Grady Booch, James Rumbaugh และ Ivar Jacobson (รู้จักกันในชื่อ "Three Amigos") เพื่อรวมแนวทางการออกแบบเชิงวัตถุที่หลากหลายให้เป็นภาษาเดียวที่ทุกคนอ่านเข้าใจตรงกัน



แผนภูมิจำแนกประเภทไดอะแกรมใน UML

กลุ่ม	ตอบคำถาม	ตัวอย่าง
Structural Diagrams (โครงสร้าง)	ระบบประกอบด้วยอะไรบ้าง	Class, Component, Deployment
Behavioral Diagrams (พฤติกรรม)	ระบบทำงาน/โต้ตอบอย่างไร	Use Case, Activity, Sequence, State Machine

Use Case Diagram อยู่ในกลุ่ม Behavioral Diagrams —

เป็นจุดเริ่มต้นที่ตอบคำถามระดับสูงสุดก่อนไดอะแกรมพฤติกรรมอื่น ๆ ทั้งหมด: "ใครใช้ระบบนี้"

และระบบต้องทำอะไรให้เขาบ้าง" ก่อนจะลงรายละเอียดต่อว่าแต่ละพฤติกรรมทำงานเป็นลำดับอย่างไร (Activity/Sequence)

หรือระบบมีสถานะอย่างไร (State Machine)

ในบริบทนี้ Use Case Diagram คือภาพรวมที่มาพร้อมกับ Use Case Lite (ข้อความบรรยาย main flow/alternate flow) ในขั้น Define/Specify — diagram บอก "ขอบเขต" ส่วนข้อความบอก "รายละเอียดการโต้ตอบ"

2 จุดประสงค์ของ Use Case Diagram

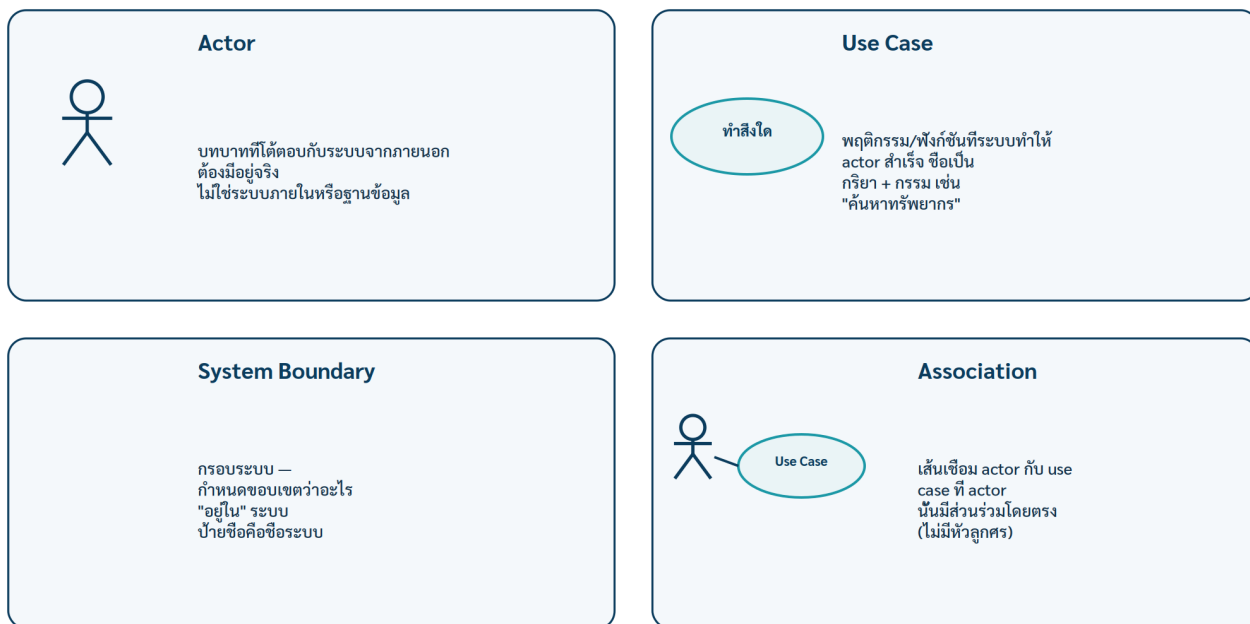
Use Case Diagram ไม่ได้มีไว้ "อธิบายทุกรายละเอียด" แต่มีไว้เพื่อ:

- สื่อสารขอบเขตระบบ — อะไรอยู่ใน/นอกระบบ (system boundary)
- ระบุผู้เกี่ยวข้องทั้งหมด — ป้องกันการมองข้าม stakeholder บางกลุ่ม
- จัดกลุ่มฟังก์ชันระดับสูง — ก่อนลงรายละเอียดเป็น flow/AC
- เป็นจุดเริ่มการเจรจาขอบเขต — คุยกับผู้เกี่ยวข้องว่า "ครบไหม" ก่อนลงมือเขียนสเปกละเอียด

สิ่งที่ Use Case Diagram ไม่ใช่

ไม่ใช่ flowchart ของขั้นตอนการทำงาน ไม่ใช่ wireframe หน้าจอ และไม่ใช่ full specification — รายละเอียดเหล่านั้นอยู่ใน Use Case (ข้อความ) และ Acceptance Criteria

3 องค์ประกอบพื้นฐาน (Basic Elements)



Legend: Actor, Use Case, System Boundary, Association

สัญลักษณ์	ความหมาย	กฎสำคัญ
Actor (รูปคนยืน)	บทบาทที่โต้ตอบกับระบบจากภายนอก	ต้องมีอยู่จริง ไม่ใช่ระบบภายในหรือฐานข้อมูล
Use Case (วงรี)	พฤติกรรม/ฟังก์ชันที่ระบบทำให้ actor สำเร็จ	ตั้งชื่อเป็น กริยา + กรรม เช่น "ค้นหาทรัพยากร"
System Boundary (กรอบ)	กำหนดขอบเขตว่าอะไร "อยู่ใน" ระบบ	ป้ายชื่อคือชื่อระบบ ไม่ใช่ชื่อโมดูลย่อย
Association (เส้นตรง)	actor มีส่วนร่วมโดยตรงกับ use case นั้น	เชื่อม actor ↔ use case เท่านั้น ไม่ใช่เชื่อม use case ↔ use case

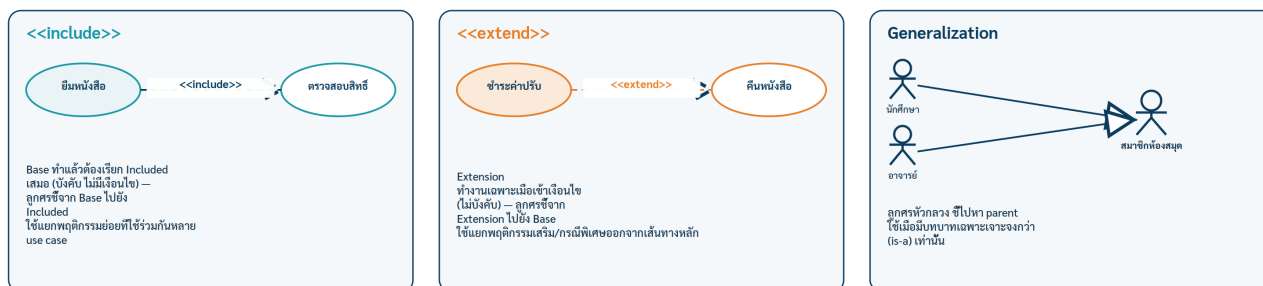
Primary Actor vs Secondary Actor

Primary Actor คือผู้ที่เริ่มการใช้งาน use case เพื่อบรรลุเป้าหมายของตนเอง (เช่น Student Requester ที่ต้องการค้นหาห้อง)

Secondary Actor คือผู้ที่ระบบต้องติดต่อยะหว่างทาง หรือมี use case ของตนเองแยกต่างหาก (เช่น เจ้าหน้าที่ที่อนุมัติคำขอ)

4 ความสัมพันธ์ (Relationships)

นอกจาก Association แล้ว ยังมีความสัมพันธ์อีก 3 แบบที่ใช้เมื่อจำเป็นเท่านั้น (ไม่ใช่ทุก diagram ต้องมี):



Legend: Include, Extend, Generalization

<<include>> — พฤติกรรมย่อยที่ใช้ร่วมกัน (บังคับเสมอ)

Use Case ต้นทาง (Base) เรียกใช้เสมอทุกครั้งที่ทำงาน ลูกศรเส้นประชี้ จาก Base ไปยัง Included

ตัวอย่าง

"ยืมหนังสือ" <<include>> "ตรวจสอบสิทธิ์การยืม" — ทุกครั้งที่ยืมหนังสือ ระบบต้องตรวจสอบสิทธิ์เสมอ ไม่มีทางเลือกอื่น

ใช้เมื่อไร: เมื่อพบพฤติกรรมย่อยเดียวกันซ้ำในหลาย use case — แยกออกมาเพื่อไม่ต้องเขียนซ้ำ

<<extend>> — พฤติกรรมเสริมที่มีเงื่อนไข (ไม่บังคับ)

Use Case ส่วนขยาย (Extension) ทำงานเฉพาะเมื่อเงื่อนไข ลูกศรเส้นประชี้ จาก Extension ไปยัง Base (ทิศทางตรงข้ามกับ include)

ตัวอย่าง

"ชำระค่าปรับ" <<extend>> "คืนหนังสือ" — การชำระค่าปรับเกิดขึ้นเฉพาะกรณีคืนหนังสือช้ากว่ากำหนดเท่านั้น ไม่ใช่ทุกครั้ง

ใช้เมื่อไร: เมื่อพบ "กรณีพิเศษ" หรือ "ทางเลือกเสริม" ที่ไม่ได้เกิดขึ้นทุกครั้ง

Generalization — บทบาทเฉพาะเจาะจงกว่า (is-a)

เส้นทึบหัวลูกศรรูปสามเหลี่ยมกลาง ชี้จาก child ไปยัง parent ใช้เมื่อ actor (หรือ use case) หนึ่งเป็น "กรณีเฉพาะ" ของอีกอันหนึ่งจริง ๆ

ตัวอย่าง

"นักศึกษา" และ "อาจารย์" ต่าง generalize มาจาก "สมาชิกห้องสมุด" — ทั้งสองมีสิทธิ์พื้นฐานเหมือนกันในฐานะสมาชิก แต่ต่างก็เป็นบทบาทเฉพาะของตนเอง

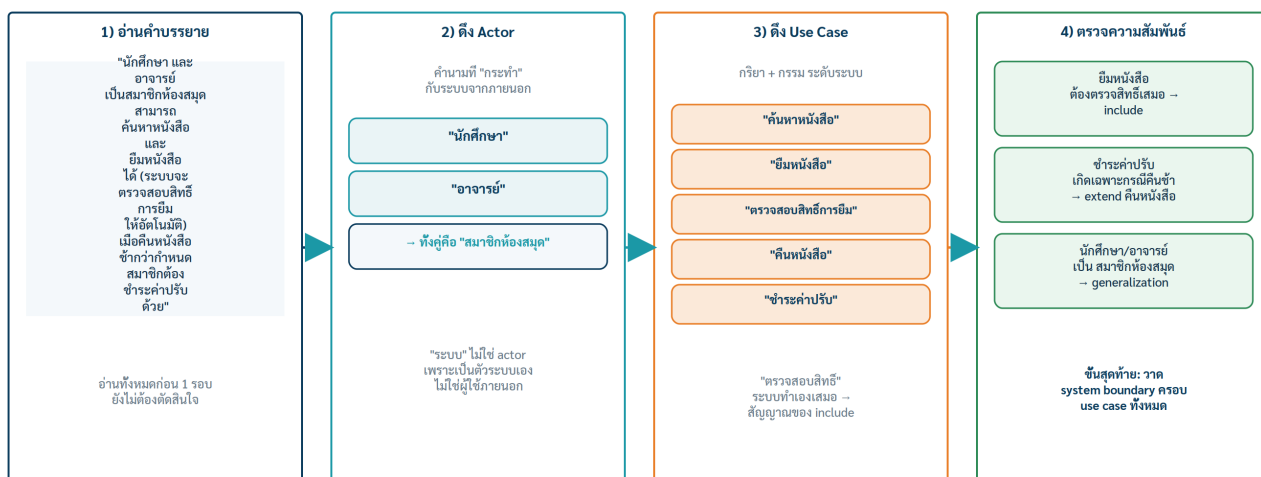
ข้อควรระวัง

อย่าใช้ include/extend/generalization พร่ำเพรื่อ — ถ้ายังไม่แน่ใจว่าจำเป็น ให้เริ่มจาก Association ธรรมดา ก่อน
ใน Core ของ Week 06 วิชานี้ ไม่บังคับใช้ความสัมพันธ์ทั้ง 3 แบบนี้เลย

5 ตัวอย่างจากสถานการณ์คุณเค: ระบบยืม-คืนหนังสือห้องสมุด

ก่อนจะไปประยุกต์กับกรณีศึกษาของวิชา เรามาฝึกวิธี "เริ่มต้นจากศูนย์" กับสถานการณ์ที่ทุกคนคุ้นเคยกันก่อน — ระบบยืม-คืนหนังสือห้องสมุด

ขั้นตอนการดึง Actor และ Use Case จากคำบรรยาย



ขั้นตอน 4 ขั้น: อ่านคำบรรยาย → ดึง Actor → ดึง Use Case → ตรวจสอบความสัมพันธ์

ขั้นที่ 1 — อ่านคำบรรยาย

"นักศึกษาและอาจารย์เป็นสมาชิกห้องสมุด สามารถค้นหาหนังสือและยืมหนังสือได้ (ระบบจะตรวจสอบสิทธิ์การยืมให้อัตโนมัติ) เมื่อคืนหนังสือช้ากว่ากำหนด สมาชิกต้องชำระค่าปรับด้วย"

ขั้นที่ 2 — ดึง Actor: ชีตเส้นใต้คำนามที่ "กระทำ" กับระบบจากภายนอก

- "นักศึกษา" และ "อาจารย์" → ทั้งคู่คือ "สมาชิกรoomสมุด" (มีบทบาทร่วมกัน จึงเป็นผู้สมัคร generalization)
- ข้อควรระวัง: "ระบบ" ไม่ใช่ actor เพราะเป็นตัวระบบเอง ไม่ใช่ผู้ใช้ภายนอก

ขั้นที่ 3 — ดึง Use Case: ชีตเส้นใต้กริยา + กรรม ระดับระบบ

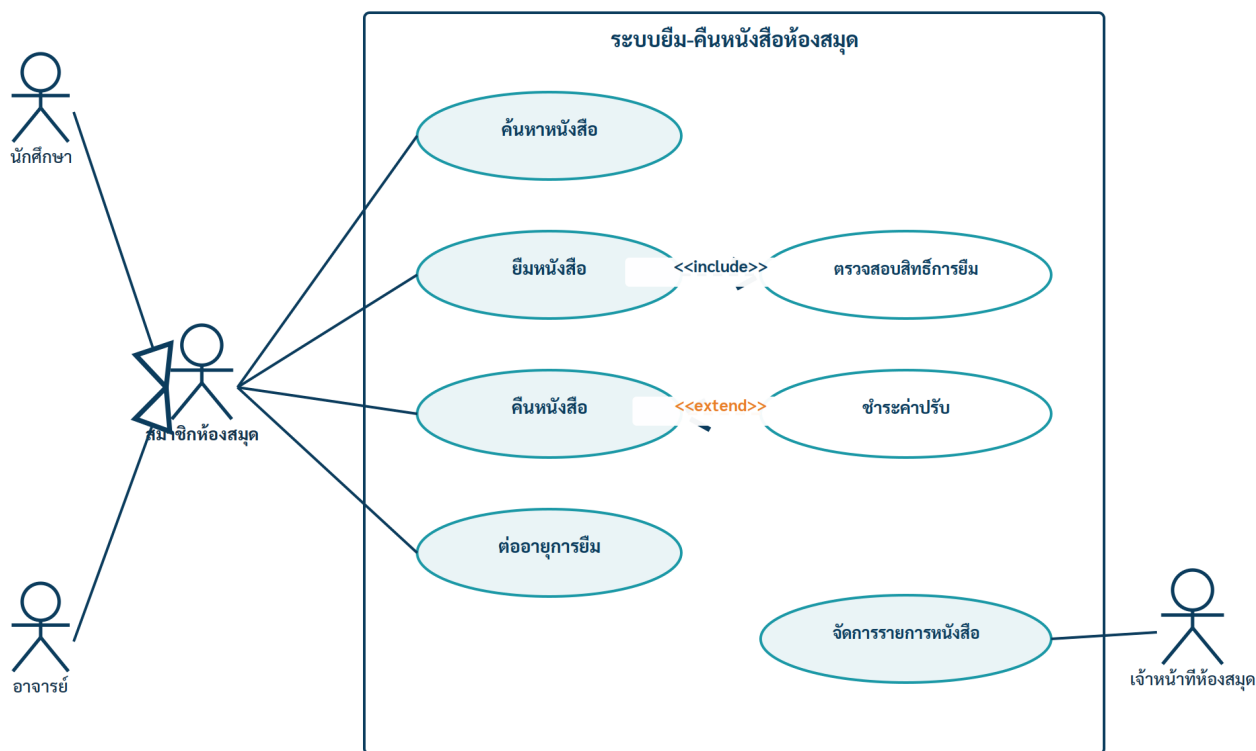
- "ค้นหาหนังสือ", "ยืมหนังสือ", "ตรวจสอบสิทธิ์การยืม", "คืนหนังสือ", "ชำระค่าปรับ"
- สัญญาณ: วลี "ระบบจะ...ให้อัตโนมัติ" มักหมายถึงพฤติกรรมที่ควรแยกเป็น use case ที่ถูก include เสมอ

ขั้นที่ 4 — ตรวจสอบความสัมพันธ์

สังเกตจากคำบรรยาย	สรุปเป็น
"ยืมหนังสือ...ระบบจะตรวจสอบสิทธิ์...ให้อัตโนมัติ" (ทำเสมอ)	ยืมหนังสือ <<include>> ตรวจสอบสิทธิ์การยืม
"เมื่อคืนช้ากว่ากำหนด...ต้องชำระค่าปรับ" (เฉพาะกรณี)	ชำระค่าปรับ <<extend>> คืนหนังสือ
"นักศึกษาและอาจารย์...เป็นสมาชิกห้องสมุด" (บทบาทร่วม)	นักศึกษา, อาจารย์ generalize → สมาชิกห้องสมุด

ขั้นสุดท้าย: วาด system boundary ครอบ use case ทั้งหมด แล้ววาง actor ไว้นอกกรอบ

ผลลัพธ์: Use Case Diagram ฉบับสมบูรณ์



Use Case Diagram ระบบยืม-คืนหนังสือห้องสมุด แสดงองค์ประกอบครบทุกแบบ

diagram นี้แสดงองค์ประกอบครบทุกแบบที่เรียนมา: Actor 3 ราย (รวม generalization), System Boundary, Use Case 7 รายการ, Association, Include และ Extend — สังเกตว่า "เจ้าหน้าที่ห้องสมุด" เป็น secondary actor ที่มี use case ของตนเอง ("จัดการรายการหนังสือ") แยกจากฝั่งสมาชิก

6 นำวิธีมาใช้กับกรณีศึกษาของเรา: Campus Resource Booking

ที่นำมาใช้วิธีการเดียวกันกับกรณีศึกษาของวิชาอื่น — Campus Resource Booking (ระบบจองทรัพยากรในมหาวิทยาลัย)

โดยเริ่มจาก Requirement Backlog ที่ผ่านมาจาก Week 05

ขั้นที่ 1-2: อ่าน Backlog แล้วดึง Actor

จาก docs/05-requirement-backlog.md — REQ-01, REQ-02, REQ-04 พูดถึง "Student Requester"

เป็นผู้ค้นหาและส่งคำขอ ส่วน REQ ที่เกี่ยวกับการอนุมัติพูดถึงบทบาท "Resource Approver" แยกต่างหาก

Actor

Student Requester, Resource Approver (ทั้งสองมีอยู่จริงในต้นทาง ไม่ได้ประดิษฐ์ขึ้นใหม่)

ขั้นที่ 3: ดึง Use Case จาก REQ ที่เลือก

REQ	กริยา + กรรม	Use Case
REQ-01	ค้นหาทรัพยากรว่าง	UC-01
REQ-02	ส่งคำขอใช้ทรัพยากร	UC-02
(ติดตามผล)	ติดตามสถานะคำขอ	UC-03
REQ-04 (ฝั่งอนุมัติ)	พิจารณาคำขอ	UC-04

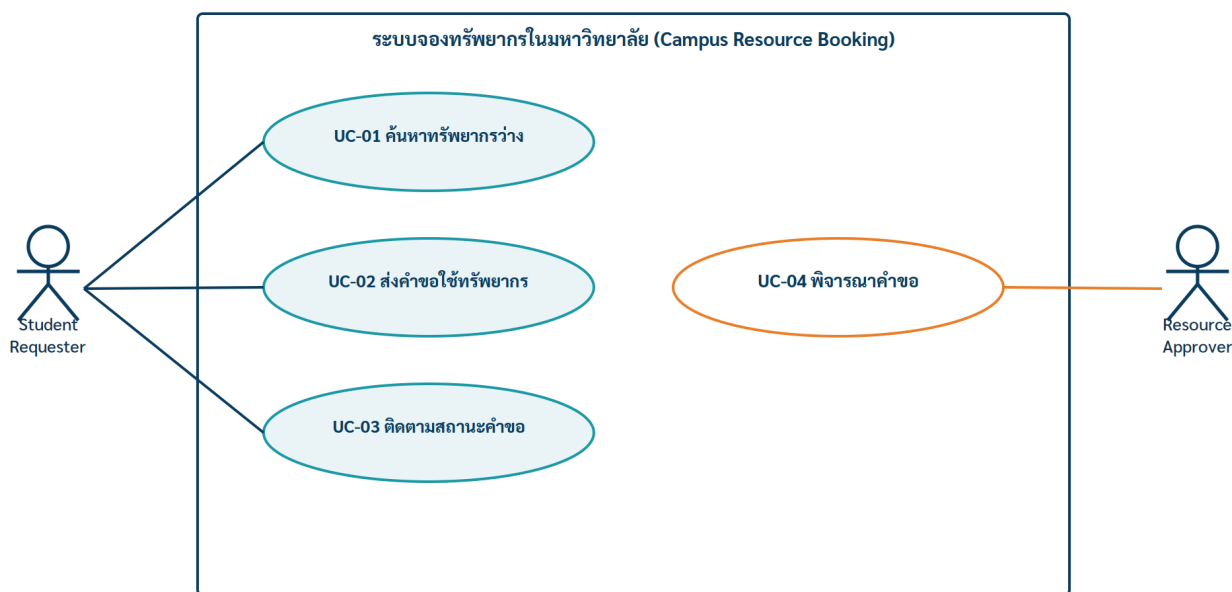
ขั้นที่ 4: ตรวจสอบความสัมพันธ์ — ผลลัพธ์คือ "ไม่มี include/extend"

ต่างจากตัวอย่างห้องสมุด เมื่อตรวจสอบ backlog ของ Core scope สัปดาห์ที่แล้ว ไม่พบพฤติกรรมย่อยที่ใช้ร่วมกันแบบบังคับ

(include) หรือกรณีเสริมแบบมีเงื่อนไข (extend) ที่จำเป็นต้องแยกออกมา — นี่คือผลลัพธ์ที่ถูกต้องเช่นกัน

เพราะความสัมพันธ์พิเศษใช้เฉพาะเมื่อจำเป็นจริง ๆ

ผลลัพธ์สุดท้าย



Use Case Diagram ของ Campus Resource Booking

diagram นี้คือ diagram มาตรฐาน (canonical) ที่ใช้เป็นตัวอย่างตลอดชุดสื่อการสอน Week 06 ของวิชานี้ — 2 actors, 1 system boundary, 4 use cases, เชื่อมด้วย association ล้วน ไม่มี include/extend เพราะอยู่ในขอบเขต Core ของสัปดาห์นี้

ข้อสังเกตสำคัญ

วิธีการ (อ่าน → ตีง actor → ตีง use case → ตรวจสอบความสัมพันธ์ → วาด boundary)

เป็นวิธีเดียวกันไม่ว่าจะใช้กับสถานการณ์ห้องสมุดหรือกรณีศึกษาจริงของวิชา — สิ่งที่แตกต่างกันคือ เนื้อหา ไม่ใช่กระบวนการ

7 ความเข้าใจผิดที่พบบ่อย

ความเข้าใจผิด	วิธีคิดที่ถูกต้อง
Diagram ต้องมี include/extend ถึงจะสมบูรณ์	ใช้เฉพาะเมื่อจำเป็นจริง — Core ของ Week 06 ไม่บังคับเลย
ยังมี use case เยอะยิ่งดี	ควรอยู่ระดับสูง (3–5 use case ต่อ actor) ไม่ใช่แตกย่อยระดับฟังก์ชันปุ่มกด
Actor ต้องเป็น "คน" เท่านั้น	Actor เป็นได้ทั้งคน ระบบภายนอก หรือเวลา (scheduler) ตราบใดที่โต้ตอบจากภายนอก
System Boundary วาดเมื่อไรก็ได้	ควรวาดหลังสุด หลังจากรู้ actor/use case/ความสัมพันธ์ครบแล้ว
Diagram แทนสเปกทั้งหมดได้	diagram บอกขอบเขต ส่วนรายละเอียด flow อยู่ใน Use Case ข้อความ
Include กับ Extend ทิศทางเหมือนกัน	ทิศทางตรงข้ามกัน — include ชี้จาก Base ไป Included, extend ชี้จาก Extension ไป Base
Generalization ใช้ได้ทุกกรณีที่ actor คล้ายกัน	ใช้เฉพาะเมื่อเป็นบทบาทเฉพาะเจาะจงกว่าจริง (is-a) ไม่ใช่แค่ "ดูคล้ายกัน"

8 แบบฝึกหัดท้ายบท

ลองใช้วิธีการ 4 ขั้นตอนในหัวข้อ 5 กับสถานการณ์ต่อไปนี้ด้วยตนเอง:

"ในการจัดกิจกรรม SE Career Talk ผู้จัดกิจกรรม (นักศึกษาคณะกรรมการ)

ต้องส่งคำขอใช้ห้องประชุมพร้อมระบุจำนวนผู้เข้าร่วม ระบบจะตรวจสอบความจุห้องให้อัตโนมัติก่อนอนุมัติ

หากผู้เข้าร่วมเกิน 100 คน ต้องขออนุมัติเพิ่มเติมจากหัวหน้าภาควิชา"

คำใบ้

- ดึง actor ออกมาก่อน (มีบทบาท มีบทบาทไหน generalize จากกันใหม่)
- ดึง use case จากกริยา+กรรม
- สังเกตคำว่า "อัตโนมัติ" (สัญญาณ include) และ "หากเกิน...ต้อง..." (สัญญาณ extend)

วาด boundary ครอบให้ครบ แล้วเทียบกับตัวอย่างห้องสมุดในหัวข้อ 5

9 สรุป

- Use Case Diagram เป็นหนึ่งใน **Behavioral Diagrams** ของ UML ใช้ตอบคำถามระดับสูงสุดว่า "ใครใช้ระบบ และระบบทำอะไรให้เขาบ้าง"
- องค์ประกอบพื้นฐาน 4 อย่าง: Actor, Use Case, System Boundary, Association
- ความสัมพันธ์พิเศษ 3 แบบ (Include, Extend, Generalization) ใช้เฉพาะเมื่อจำเป็นจริง ไม่ใช่ทุก diagram ต้องมี
- วิธีเริ่มต้นจากศูนย์: อ่านคำบรรยาย → ตีง Actor → ตีง Use Case → ตรวจสอบความสัมพันธ์ → วาด Boundary ครอบ
- วิธีการเดียวกันนี้ใช้ได้ทั้งกับสถานการณ์ทั่วไป (ห้องสมุด) และกรณีศึกษาจริงของวิชา (Campus Resource Booking)
- Diagram บอก **ขอบเขต** ไม่ใช่รายละเอียดพฤติกรรม — รายละเอียดอยู่ใน Use Case ข้อความ (main/alternate flow) ที่ต่อยอดจาก diagram นี้

เอกสารนี้เป็นส่วนหนึ่งของชุดสื่อการสอน ENGSE206 Week 06 — ใช้ประกอบคู่กับเอกสาร "User Story — ทฤษฎี หลักการ และการแปลงจากสถานการณ์จริง"